

Smart India Hackathon (SIH26151: NTRO)

System Architecture & Completed Implementation Walkthrough (Steps 1 - 4)

Step	Module Name	Core Capability Delivered
Step 1	Evidence Collection	Scrapes BTC, ETH, XMR wallets, handles, PGP keys; logs SHA-256 hashes.
Step 2	Storage & Entity Resolution	Multi-pivot correlation merging fragmented aliases into unified Actor UUIDs.
Step 3	Correlation Engine	Bipartite graph mapping (NetworkX) detecting co-conspirator syndicates.
Step 4	OPSEC & Infra Analysis	Murmur3 favicon hashing & HTML DOM skeleton checks identifying clone sites.

Step 1 - Evidence Collection (entity_extractor.py)

Threat actors deliberately disguise their traces. This module parses raw, unstructured HTML from dark web platforms. Using regular expressions and DOM sanitization, it extracts Bitcoin (Legacy/Bech32), Ethereum, Monero addresses, PGP key blocks, and communication handles (Jabber, Telegram, TOX). To ensure court admissibility, every capture generates a cryptographic SHA-256 content checksum and UTC timestamp to prove data integrity.

Step 2 - Evidence Storage & Deterministic Resolution (resolution_pipeline.py)

Criminals operate under rotating pseudonyms across separate forums (e.g., DreadOps on Dread vs. ApexBreach on Exploit). This module manages a relational schema and executes multi-pivot entity resolution. When new evidence arrives, it evaluates overlapping wallets, handles, or PGP keys. If any identifier matches, the engine links the new alias directly to the existing master Actor UUID, preventing fragmented profiles and establishing shared attribution automatically.

Step 3 - Evidence Correlation Engine (correlation_engine.py)

Identities are rarely isolated in single-step relationships. Using NetworkX, this module constructs an undirected bipartite network connecting actors to their shared financial and communication footprints. Connected component algorithms automatically cluster co-conspirators into criminal syndicates, while shortest-path algorithms trace forensic linkages across multiple hops.

Step 4 - OPSEC & Infrastructure Analysis (opsec_analyzer.py)

Even when threat actors change usernames and contact information, they often reuse web infrastructure. This module profiles onion sites through two technical vectors: (1) Shodan-standard MurmurHash3 favicon hashing, and (2) HTML DOM skeleton structural hashing. By matching these fingerprints across different .onion hidden services, the engine exposes backend template reuse and shared server hosting.

Verification Status: All 4 modules have been validated locally inside the project environment. Threat actor resolution was verified via FastAPI Swagger endpoints, network links were rendered in PyVis, and OPSEC leaks were identified between separate onion services.